Disciplina: **Teste de Software**

Refatoração de Testes e Detecção de Test Smells

Aluno: João Gabriel Maia da Costa

Matrícula: 837351

Repositório: https://github.com/Joaogabrielmaia/test-smelly

1. Análise de Smells

A análise manual do arquivo `userService.smelly.test.js` revelou os três smells abaixo (mais um quarto, o teste ignorado, apontado também pelo linter).

1.1 Lógica Condicional no Teste (Conditional Test Logic)

O teste "deve desativar usuários se eles não forem administradores" usa `for` e `if/else` para percorrer um usuário comum e um administrador. **Por que é um mau cheiro:** o próprio teste passa a ter lógica que também precisaria ser testada, e cada execução só exercita um ramo por usuário. **Risco:** se a lista mudar (ou `isAdmin` vier errado), um ramo deixa de rodar e nenhuma asserção falha; além disso, quando falha, não fica claro qual cenário quebrou.

1.2 Teste que Engole a Falha (try/catch sem falha) / Asserção Condicional

O teste "deve falhar ao criar usuário menor de idade" coloca o `expect` dentro do `catch`. **Por que é um mau cheiro:** se nenhuma exceção for lançada, o `catch` nunca executa e o teste passa sem verificar nada. **Risco:** falso positivo. Se alguém remover a validação `idade < 18` do serviço, a suíte continua verde e o bug chega à produção (confirmado: removi a validação e o teste antigo continuou passando).

1.3 Teste Frágil (Fragile Test) e Eager Test

O teste do relatório compara a linha exata `ID: ..., Nome: Alice, Status: ativo\n` e o prefixo do título. O comentário do código-fonte avisa que "o formato do relatório pode mudar". **Risco:** qualquer mudança cosmética (espaço, ordem dos campos, título) quebra o teste sem haver bug de comportamento, gerando manutenção constante e desconfiança na suíte. O primeiro teste ("criar e buscar") também é *Eager*: valida criação e busca juntas, com dois "Acts"; se falhar, não se sabe qual operação causou o problema.

1.4 Teste Ignorado / Sem Asserções

`test.skip` com apenas um TODO: o cenário de lista vazia nunca é verificado, criando falsa sensação de cobertura.

2. Processo de Refatoração

Teste escolhido: o da desativação (o mais problemático: loop, `if/else` e asserções condicionais).

Antes (smelly)

```
test('deve desativar usuários se eles não forem administradores', () => {
  const usuarioComum = userService.createUser('Comum', 'comum@teste.com', 30);
  const usuarioAdmin = userService.createUser('Admin', 'admin@teste.com', 40, true);
  const todosOsUsuarios = [usuarioComum, usuarioAdmin];
  for (const user of todosOsUsuarios) {
    const resultado = userService.deactivateUser(user.id);
    if (!user.isAdmin) {
      expect(resultado).toBe(true);
      const usuarioAtualizado = userService.getUserById(user.id);
      expect(usuarioAtualizado.status).toBe('inativo');
    } else {
      expect(resultado).toBe(false);
    }
  }
});
```

Depois (clean)

```
describe('deactivateUser', () => {
  test('retorna true ao desativar um usuário comum', () => {
    const usuarioComum = userService.createUser('Comum', 'comum@teste.com', 30); // Arrange

    const resultado = userService.deactivateUser(usuarioComum.id);               // Act
```

```
    expect(resultado).toBe(true);                                                    // Assert
  });

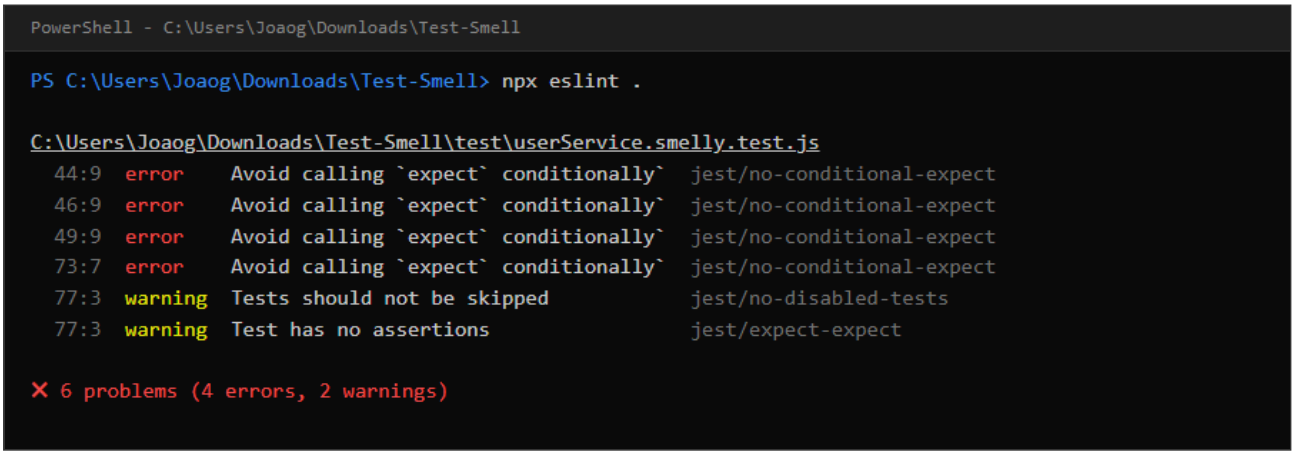
  test('altera o status do usuário comum para inativo', () => { /* AAA */ });
  test('retorna false ao tentar desativar um administrador', () => { /* AAA */ });
  test('mantém o administrador ativo após a tentativa de desativação', () => { /* AAA */ });
  test('retorna false quando o usuário não existe', () => { /* AAA */ });
});
```

Decisões tomadas

Smell	Correção aplicada
Lógica condicional	Removidos `for`/`if`: cada cenário (comum, admin, inexistente) virou um teste próprio, com uma única razão para falhar.
try/catch silencioso	Uso de `expect(fn).toThrow('mensagem')`, que falha se nenhuma exceção ocorrer. Validado com um "mutante" (validação de idade removida): o teste antigo passa, o novo falha. Campos obrigatórios usam `test.each`.
Fragilidade	O relatório é verificado por conteúdo (contém id, nome e status do usuário), não pelo formato exato da linha, espaços ou quebras de linha.
Eager Test	"Criar e buscar" foi dividido em testes de `createUser` e de `getUserById`, agrupados em `describe` por método.
Teste ignorado	Implementado o cenário de lista vazia ("Nenhum usuário cadastrado.").
Nomes / AAA	Nomes descrevem o comportamento esperado; todos seguem Arrange, Act, Assert separados por linha em branco. Também foram cobertos limite de idade (18), campos ausentes, id inexistente e unicidade de ids.

3. Relatório da Ferramenta (ESLint)

Configuração: `eslint@8` + `eslint-plugin-jest`, com o `.eslintrc.json` do enunciado. Resultado da primeira execução (`npx eslint .`):



A ferramenta detectou automaticamente a asserção condicional do loop/`if` (linhas 44, 46, 49), o `expect` dentro do `catch` (linha 73) e o teste ignorado e vazio (linha 77), em segundos, sem executar os testes, apontando localização exata e regra violada. Por outro lado, ela **não** detecta smells semânticos como o Teste Frágil (formato exato do relatório) ou o Eager Test, que exigem análise manual: as duas abordagens se complementam. Após a refatoração, `npx eslint __tests__` retorna zero erros e zero avisos, e `npm test` executa 2 suítes: 27 testes passando e 1 ignorado (o `test.skip` do arquivo original, mantido intacto).

4. Conclusão

Este trabalho mostrou que um teste verde não é necessariamente um bom teste: a suíte original passava mesmo com a regra de maioridade removida do código. Testes limpos (AAA, um comportamento por teste, sem lógica condicional, nomes descritivos) funcionam como documentação executável, falham pelo motivo certo e indicam rapidamente onde está o defeito. A análise estática com ESLint automatiza a detecção de parte dos smells de forma barata e repetível, podendo ser integrada ao CI para impedir que maus cheiros voltem. Juntos, isso reduz o custo de manutenção, aumenta a confiança nas mudanças e torna o projeto mais sustentável ao longo do tempo.